VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Hitha Harish (1BM23CS115)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Jan-2026 to May-2026

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Machine Learning (23CS6PCMAL)" carried out by **Hitha Harish (1BM23CS115),** who is bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Dr. Adarsha Sagar H V Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

Sl. No.	Date	Experiment Title	Page No.
1	04-02-2026	Analysis of Iris Dataset	4 - 8
2	11-02-2026	House Price Prediction Dataset Analysis	9 - 15
3	25-02-2026	KNN Algorithm	16 - 18
4	11-03-2026	Build Logistic Regression Model for a given dataset	19 - 21
5	25-03-2026	Decision Tree Classification & Regression	22 - 25
6	01-04-2026	SVM	26 - 27
7	01-04-2026	Random Forest	28 - 29
8	01-04-2026	K - Means	30 -31
9	01-04-2026	ADA Boost	32 - 33
10	08-04-2026	Principal Component Analysis (PCA)	34 - 35

Github Link : https://github.com/HithaHarishCS/ML/

Program 1

Analysis of a Iris Dataset

Screenshot :

papergrid

LAB - 1

Date: 4 / 2 / 25

3 a) CODE :

```
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns = iris.feature_names)
df['species'] = iris.target
df.head()
```

o/p: ~~sepal length (cm) sepal width (cm) petal length (cm)~~

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Question How many columns are there.

5 columns in total - 4 input & 1 output column

3 b) df.shape

& c df.info()

df.columns

o/p: (150, 5)

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
```

#	Column	Non-Null Count	Dtype
0	sepal length (cm)	150 non-null	float64
1	sepal width (cm)	150 non-null	float64
2	petal length (cm)	150 non-null	float64
3	petal width (cm)	150 non-null	float64
4	species	150 non-null	int64

```
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

papergrid

Date: / /

```
Index(['sepal width (cm)', 'sepal length (cm)', 'petal length (cm)', 'petal width (cm)', 'species'],
      dtype='object')
```

Questions: How many rows (examples)?

150

How many features?

4

Which column is the output?

species.

3d) df.describe()

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
count	150.000	150.000	150.000	150.000	150.000
mean	5.843	3.057	3.758	1.199	1.000
std	0.828	0.435	1.765	0.762	0.819
min	4.300	2.000	1.000	0.100	0.000
25%	5.100	2.800	1.600	0.300	0.000
50%	5.800	3.000	4.350	1.300	1.000
75%	6.400	3.300	5.100	1.800	2.000
max	7.900	4.400	6.900	2.500	2.000

Which feature has largest values?

sepal length.

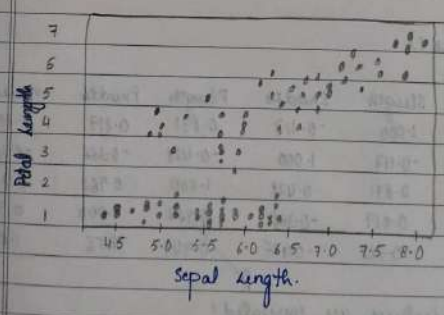
3c) df ['species'].value_counts()

df species	count
0	50
1	50
2	50

dtype: int64

Are classes balanced?
Yes.

3f) import matplotlib.pyplot as plt
plt.scatter(df ['sepal length(cm)'], df['petal length (cm)'])
plt.xlabel ("sepal length")
plt.ylabel ("Petal Length")
plt.show()



Do you see groups forming?
Yes.

3g) df [df ['species'] == 0].head()
df [df ['species'] == 1].head().

	sepal length	sepal width	petal length	petal width	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

	Slength	Swidth	Plength	Pwidth	species
50	7.0	3.2	4.7	1.4	1
51	6.4	3.2	4.5	1.5	1
52	6.9	3.1	4.9	1.5	1
53	5.5	2.3	4.0	1.3	1
54	5.5	2.8	4.6	1.5	1

Do features values differ?
Yes.

3h) df.corr()

	Slength	Swidth	Plength	Pwidth	species
Slength	1.000	-0.117	0.871	0.817	0.782
Swidth	-0.117	1.000	-0.428	-0.366	-0.426
Plength	0.871	-0.428	1.000	0.962	0.949
Pwidth	0.817	-0.366	0.962	1.000	0.956
species	0.782	-0.426	0.949	0.956	1.000

Which features are correlated?
Petal length & Petal length.
Sepal length & Sepal width

3i) df.sort_values (by = 'petal length (cm)').head()

	sepal L	sepal W	petal L	petal W	species
22	4.6	3.6	1.0	0.2	0
13	4.3	3.6	1.1	01	0
14	5.8	3.0	1.2	0.2	0
35	5.0	4.0	1.2	0.2	0
16	5.4	3.2	1.3	0.4	0

What happens to species label?
gets reordered, no data loss.

04/03/2026

Code :

```
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target
df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
df.shape
```

```
(150, 5)
```

```
df.columns
```

```
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
       'petal width (cm)', 'species'],
      dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   species            150 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

```
df.describe()
```

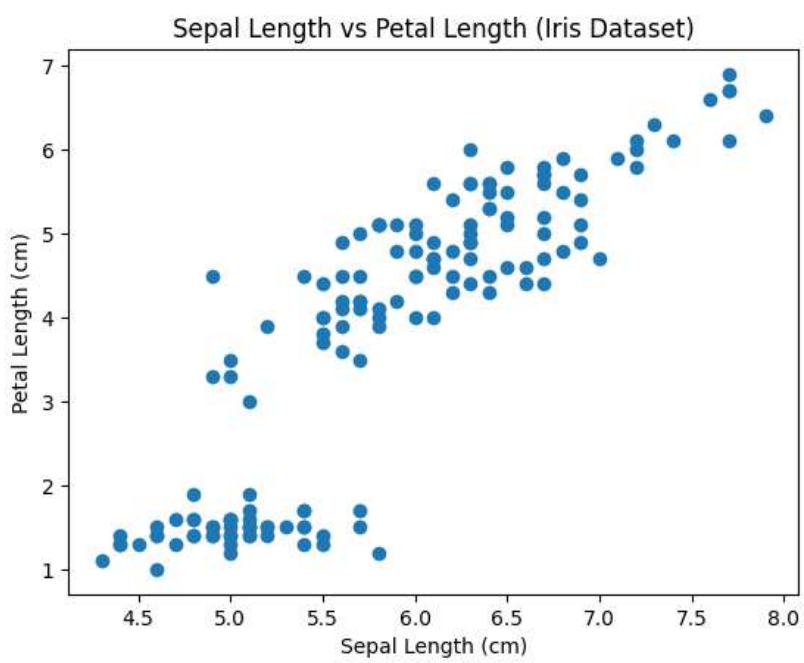
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

```python
df['species'].value_counts()
```

species	count
0	50
1	50
2	50

dtype: int64

```python
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

# Scatter plot: Sepal Length vs Petal Length
plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Petal Length (cm)")
plt.title("Sepal Length vs Petal Length (Iris Dataset)")
plt.show()
```



```
df[df['species'] == 0].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
df[df['species'] == 1].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
50	7.0	3.2	4.7	1.4	1
51	6.4	3.2	4.5	1.5	1
52	6.9	3.1	4.9	1.5	1
53	5.5	2.3	4.0	1.3	1
54	6.5	2.8	4.6	1.5	1

```
df.corr()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
sepal length (cm)	1.000000	-0.117570	0.871754	0.817941	0.782561
sepal width (cm)	-0.117570	1.000000	-0.428440	-0.366126	-0.426658
petal length (cm)	0.871754	-0.428440	1.000000	0.962865	0.949035
petal width (cm)	0.817941	-0.366126	0.962865	1.000000	0.956547
species	0.782561	-0.426658	0.949035	0.956547	1.000000

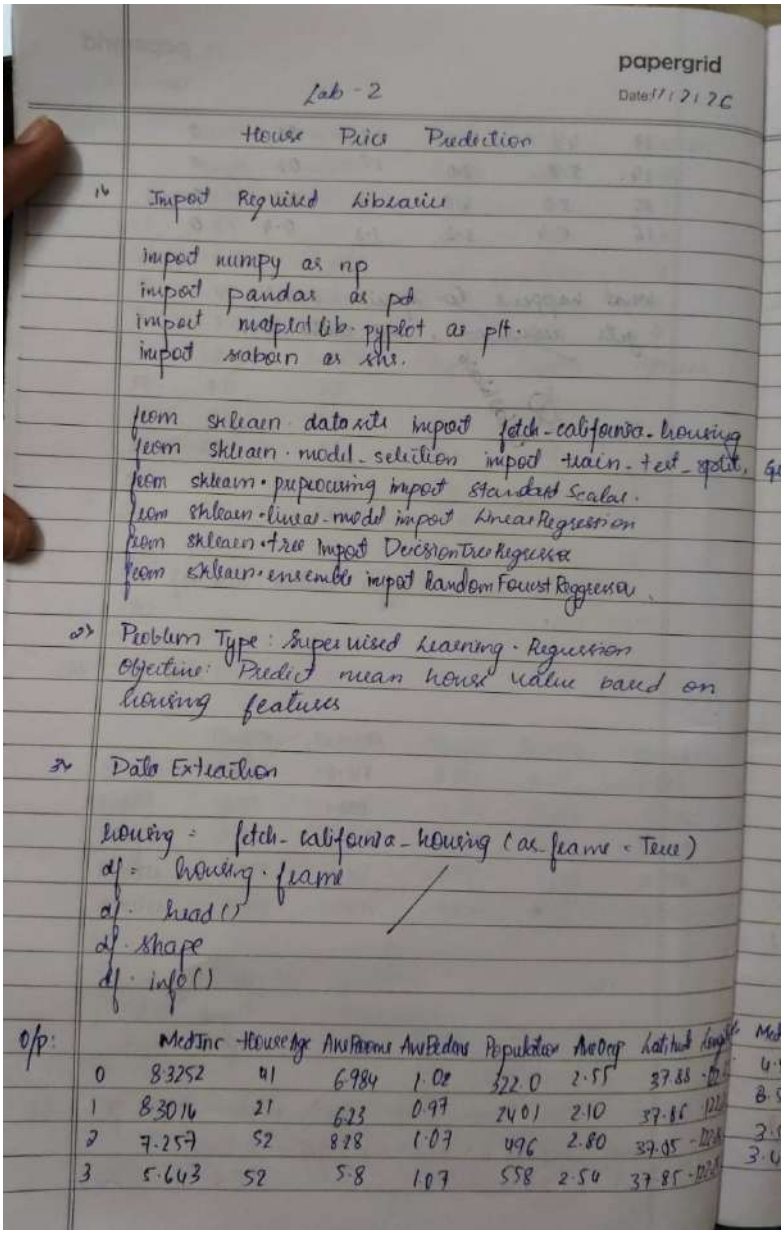
```
df.sort_values(by='petal length (cm)').head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
22	4.6	3.6	1.0	0.2	0
13	4.3	3.0	1.1	0.1	0
14	5.8	4.0	1.2	0.2	0
35	5.0	3.2	1.2	0.2	0
16	5.4	3.9	1.3	0.4	0

Program 2

House Price Prediction Datasets

Screenshot :



papergrid

Lab - 2 Date: 11/2/26

House Price Prediction

1) Import Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt.
import seaborn as sns.

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split,
from sklearn.preprocessing import StandardScaler.
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor.
```

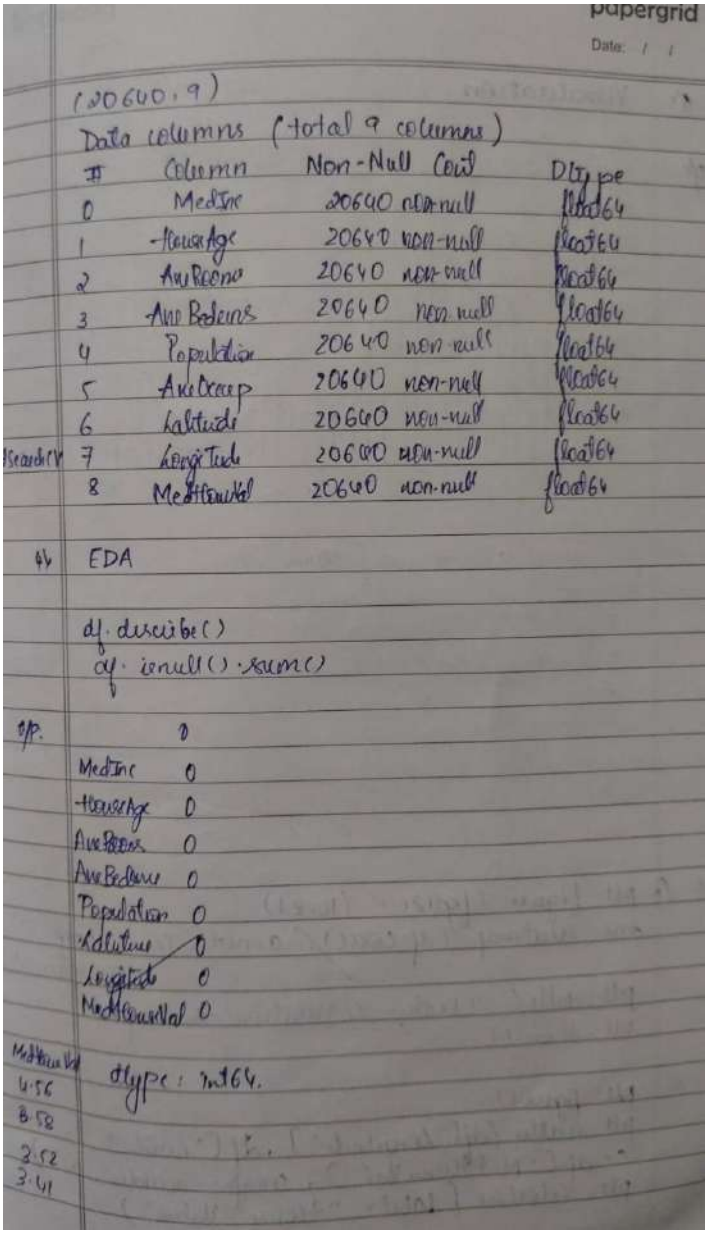
2) Problem Type : Supervised Learning - Regression
Objective: Predict mean house value based on housing features

3) Data Extraction

```
housing = fetch_california_housing(as_frame = True)
df = housing.frame
df.head()
df.shape
df.info()
```

o/p:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOcc	Latitude	Longitude	Med
0	8.3252	41	6.984	1.02	322.0	2.55	37.88	-122	4.
1	8.3014	21	6.23	0.97	2401	2.10	37.86	122	8.
2	7.257	52	8.28	1.07	496	2.80	37.85	-122	3.
3	5.643	52	5.8	1.07	558	2.54	37.85	-122	3.



papergrid

Date: / /

(20640, 9)

Data columns (total 9 columns)

#	Column	Non-Null Count	Dtype
0	MedInc	20640 non-null	float64
1	HouseAge	20640 non-null	float64
2	AveRooms	20640 non-null	float64
3	AveBedrms	20640 non-null	float64
4	Population	20640 non-null	float64
5	AveOccup	20640 non-null	float64
6	Latitude	20640 non-null	float64
7	Longitude	20640 non-null	float64
8	MedHouseVal	20640 non-null	float64

4) EDA

```
df.describe()
df.isnull().sum()
```

o/p.

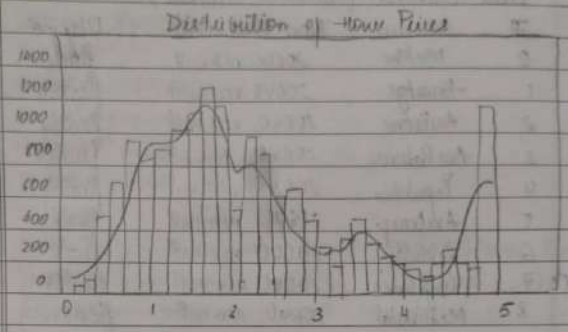
	0
MedInc	0
HouseAge	0
AveRooms	0
AveBedrms	0
Population	0
AveOccup	0
Latitude	0
Longitude	0
MedHouseVal	0

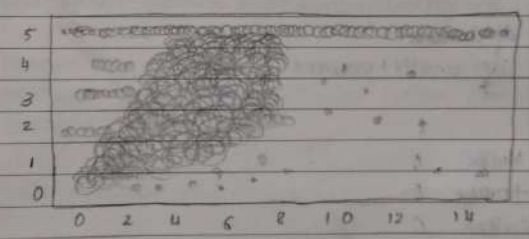
MedHouseVal
4.56
8.58
3.52
3.41

dtype: int64.

5) Visualization

o/p:





CODE: ① plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Feature Correlation Heatmap")
plt.show()

② plt.figure()
plt.scatter(df['Longitude'], df['Latitude'], c=df['MedHouseVal'], cmap='viridis', s=5)
plt.colorbar(label='House Value')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.title('House Prices by Location')
plt.show()

5) Prepare the Data

6.1) Feature Engineering
df['RoomsPerHousehold'] = df['AveRooms'] / df['HouseAge']

6.2) Train-Test Split
x = df.drop('MedHouseVal', axis=1)
y = df['MedHouseVal']
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

6.3) Feature Scaling

scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)

8) Train Advanced Models:

8.1) Decision Tree Regressor.

dt = DecisionTreeRegressor(random_state=42)
dt.fit(x_train, y_train)
y_pred_dt = dt.predict(x_test)
rmse_dt = np.sqrt(mean_squared_error(y_test, y_pred_dt))

rmse_dt

o/p: np.float64(0.713)

Date: / /

8) Random Forest Regressor

```
rf = RandomForestRegressor(random_state=42)
rf.fit(x_train, y_train)
y_pred_rf = rf.predict(x_test)
rmse_rf = np.sqrt(mean_squared_error(
                y_test, y_pred_rf))
rmse_rf.
```

O/P np.float64(0.512)

9) Select & Train a Baseline Model.

9.1 Linear Regression

```
lin_reg = LinearRegression()
lin_reg.fit(x_train_scaled, y_train)
y_pred_lr = lin_reg.predict(x_test_scaled)
rmse_lr = np.sqrt(mean_squared_error(y_test,
                y_pred_lr))
mae_lr = mean_absolute_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)
rmse_lr, mae_lr, r2_lr.
```

(np.float64(0.746), 0.55, 0.576)

9) Fine Tune the Model

```
param_grid = {'n_estimators': [50, 100], 'max_depth':
[None, 10, 20], 'min_samples_split': [2, 5]}

grid_search = GridSearchCV(rf, param_grid, cv=3,
scoring='neg_mean_squared_error')

grid_search.fit(x_train, y_train)
```

Date: / /

```
best_model = grid_search.best_estimator_
y_pred_best = best_model.predict(x_test)
rmse_best = np.sqrt(mean_squared_error
                (y_test, y_pred_best))
rmse_best.
```

9) Model Comparison

```
results = pd.DataFrame({'Model': ['Linear Regression', 'Decision Tree', 'Random Forest', 'Tuned
Random Forest'], 'RMSE': [rmse_lr, rmse_dt,
rmse_rf, rmse_best]})

results.
```

10) 1) Which Model Performed best?
Tuned Random Forest Regressor.

2) Why did Random Forest outperform Linear Regression?
captures non-linear relationships & reduces overfitting.

3) How did tuning improve performance?
- prevented overfitting
- improved generalization

4) What additional data could further improve prediction?
- crime data
- school quality
- distance to city centers.

Income

Code :

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

Problem Type: Supervised Learning – Regression Objective: Predict median house value using housing features. Evaluation Metrics: RMSE, MAE, R^2 score.

In [2]:
```
# Load dataset
housing = fetch_california_housing(as_frame=True)

# Convert to dataframe
df = housing.frame

# View data
df.head()
```

Out[2]:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422

In [3]:
```
df.shape
```

Out[3]: (20640, 9)

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   MedInc       20640 non-null  float64
 1   HouseAge     20640 non-null  float64
 2   AveRooms     20640 non-null  float64
 3   AveBedrms    20640 non-null  float64
 4   Population   20640 non-null  float64
 5   AveOccup     20640 non-null  float64
 6   Latitude     20640 non-null  float64
 7   Longitude    20640 non-null  float64
 8   MedHouseVal  20640 non-null  float64
dtypes: float64(9)
memory usage: 1.4 MB
```

File display

```
df.describe()
```

Out[5]:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVa
count	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.00000(
mean	3.870671	28.639486	5.429000	1.096675	1425.476744	3.070655	35.631861	-119.569704	2.068558
std	1.899822	12.585558	2.474173	0.473911	1132.462122	10.386050	2.135952	2.003532	1.153956
min	0.499900	1.000000	0.846154	0.333333	3.000000	0.692308	32.540000	-124.350000	0.14999(
25%	2.563400	18.000000	4.440716	1.006079	787.000000	2.429741	33.930000	-121.800000	1.19600(
50%	3.534800	29.000000	5.229129	1.048780	1166.000000	2.818116	34.260000	-118.490000	1.79700(
75%	4.743250	37.000000	6.052381	1.099526	1725.000000	3.282261	37.710000	-118.010000	2.64725(
max	15.000100	52.000000	141.909091	34.066667	35682.000000	1243.333333	41.950000	-114.310000	5.00001(

In [6]:

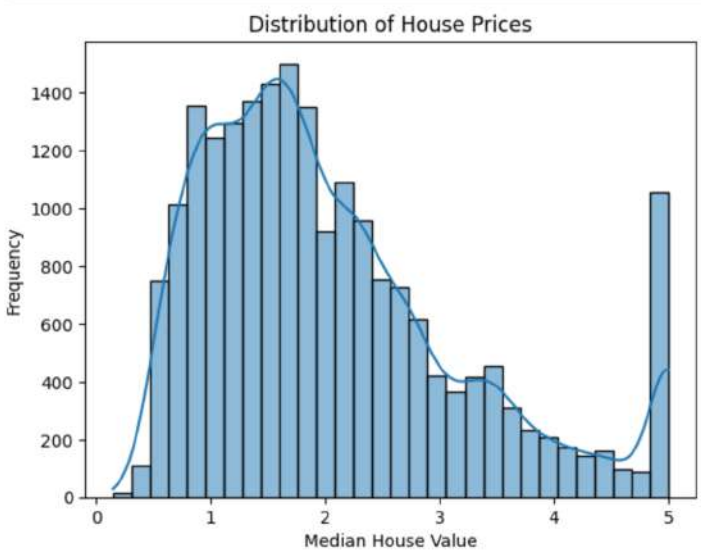
```
df.isnull().sum()
```

Out[6]:

	0
MedInc	0
HouseAge	0
AveRooms	0
AveBedrms	0
Population	0
AveOccup	0
Latitude	0
Longitude	0
MedHouseVal	0

In [7]:

```
plt.figure()
sns.histplot(df['MedHouseVal'], bins=30, kde=True)
plt.xlabel("Median House Value")
plt.ylabel("Frequency")
plt.title("Distribution of House Prices")
plt.show()
```

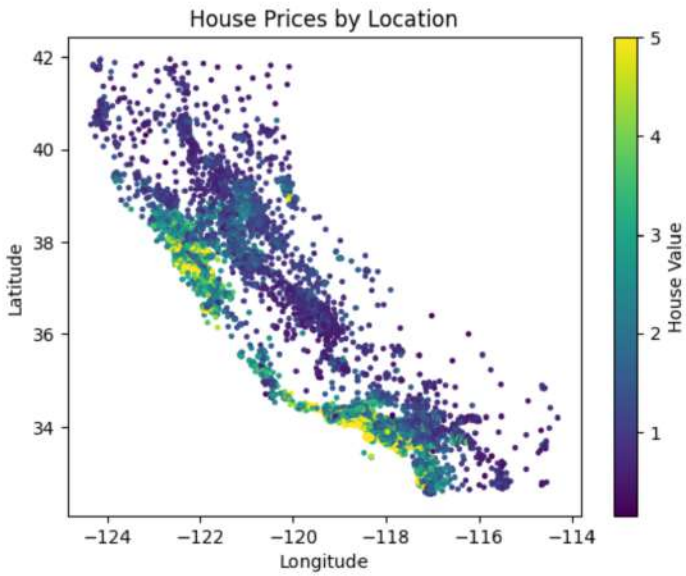


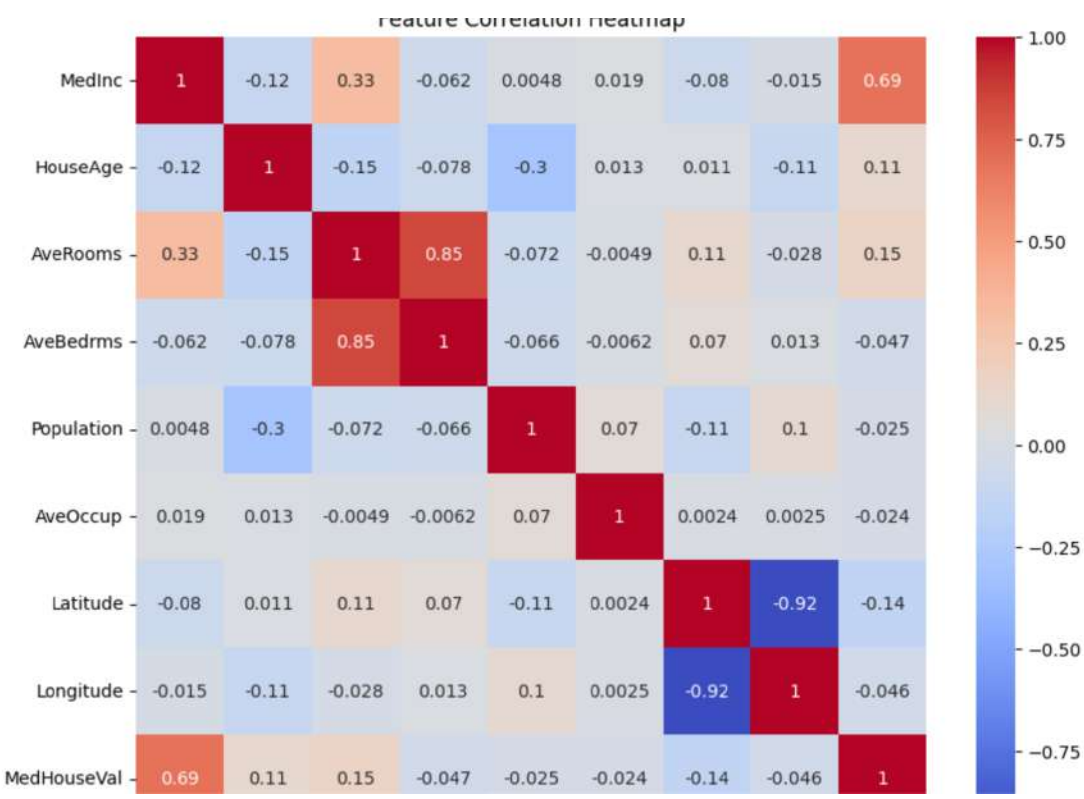
In [8]:

```
plt.figure()
sns.scatterplot(x=df['MedInc'], y=df['MedHouseVal'], alpha=0.5)
plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.title("Income vs House Price")
```

In [10]:

```
plt.figure()
plt.scatter(df['Longitude'], df['Latitude'],
            c=df['MedHouseVal'], cmap='viridis', s=5)
plt.colorbar(label="House Value")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.title("House Prices by Location")
plt.show()
```





In [19]:

```
results = pd.DataFrame({
    'Model': ['Linear Regression', 'Decision Tree', 'Random Forest', 'Tuned Random Forest'],
    'RMSE': [rmse_lr, rmse_dt, rmse_rf, rmse_best]
})

results
```

Out[19]:

	Model	RMSE
0	Linear Regression	0.744954
1	Decision Tree	0.713838
2	Random Forest	0.512864
3	Tuned Random Forest	0.513263

Program 3

KNN Algorithm :

Screenshot :

LAB-3

KNN Algorithm

Input:
Training Dataset D
Test sample x
Number of neighbours K

Output:
Predicted class for x.

Algorithm:

1) For each training sample x_i in dataset D,
Compute distance between x & x_i
2) Sort all training samples by distance
3) Select the first K nearest neighbours
4) Count the class labels of these K neighbours
5) Assign x to the class with the highest frequency.
6) Return the predicted class.

CODE:-

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score.

iris = load_iris()
x = iris.data
y = iris.target

x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size = 0.2, random_state = 42)

knn = KNeighborsClassifier(n_neighbors = 3)
knn.fit(x_train, y_train)
y_pred = knn.predict(x_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
sample = [[5.1, 3.5, 1.4, 0.2]]
print("Predicted class:", knn.predict(sample))
```

O/P: Accuracy : 1.0
Predicted class : [0]

Multilinear Regression

Algorithm.

Input:
Dataset with multiple independent variables x_1, x_2, x_n
& dependent variable y

Output:
Predicted value $\hat{y}$

Algorithm
1) Load the dataset
2) Separate dataset into training & testing sets
3) split dataset into training & testing sets
4) create Multi Linear Regression model
5) train the model using training data
6) predict y values for test data
7) Evaluate model
8) Return predicted value.

CODE:

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

data = load_diabetes()
x = data.data
y = data.target

x_train, x_test, y_train, y_test = train_test_split(x, y, test_
size = 0.2, random_state = 42)
model = LinearRegression()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
print("Sample Predictions:", y_pred[5])
```

O/P: Sample Predictions : [139.5475, 179.5172, 134.0387,
291.4170 123.7896]

Code:

In [1]:

```
File display
om google.colab import files
uploaded = files.upload()
```

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Iris.csv to Iris.csv
```

In [2]:

```
# ================================
# IMPORTS
# ================================
import numpy as np
import pandas as pd

# ================================
# LOAD DATASET
# ================================
df = pd.read_csv("Iris.csv")   # change name if needed

print("Dataset Preview:")
print(df.head())

# ================================
# ENCODE LABELS (STRING → NUMBER)
# ================================
def encode_labels(column):
    unique_vals = list(set(column))
    mapping = {val: i for i, val in enumerate(unique_vals)}
    encoded = np.array([mapping[val] for val in column])
    return encoded, mapping

# ================================
# NORMALIZE DATA
# ================================
def normalize(X):
    return (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))

# ================================
# TRAIN TEST SPLIT
# ================================
def train_test_split(X, y, test_size=0.2):
    indices = np.arange(len(X))
    np.random.shuffle(indices)

    split = int(len(X) * (1 - test_size))

    return X[indices[:split]], X[indices[split:]], y[indices[:split]], y[indices[split:]]

# ================================
File display
# D NCTION
# ================================
def euclidean_distance(a, b):
    return np.sqrt(np.sum((a - b) ** 2))

# ================================
# KNN ALGORITHM
# ================================
def knn_predict(X_train, y_train, X_test, k=3):
    predictions = []

    for test_point in X_test:
        distances = []

        for i in range(len(X_train)):
            dist = euclidean_distance(test_point, X_train[i])
            distances.append((dist, y_train[i]))

        distances.sort(key=lambda x: x[0])
        neighbors = [label for _, label in distances[:k]]

        prediction = max(set(neighbors), key=neighbors.count)
        predictions.append(prediction)

    return np.array(predictions)

# ================================
# ACCURACY
# ================================
def accuracy(y_true, y_pred):
    return np.sum(y_true == y_pred) / len(y_true)

# ================================
# CONFUSION MATRIX
# ================================
```

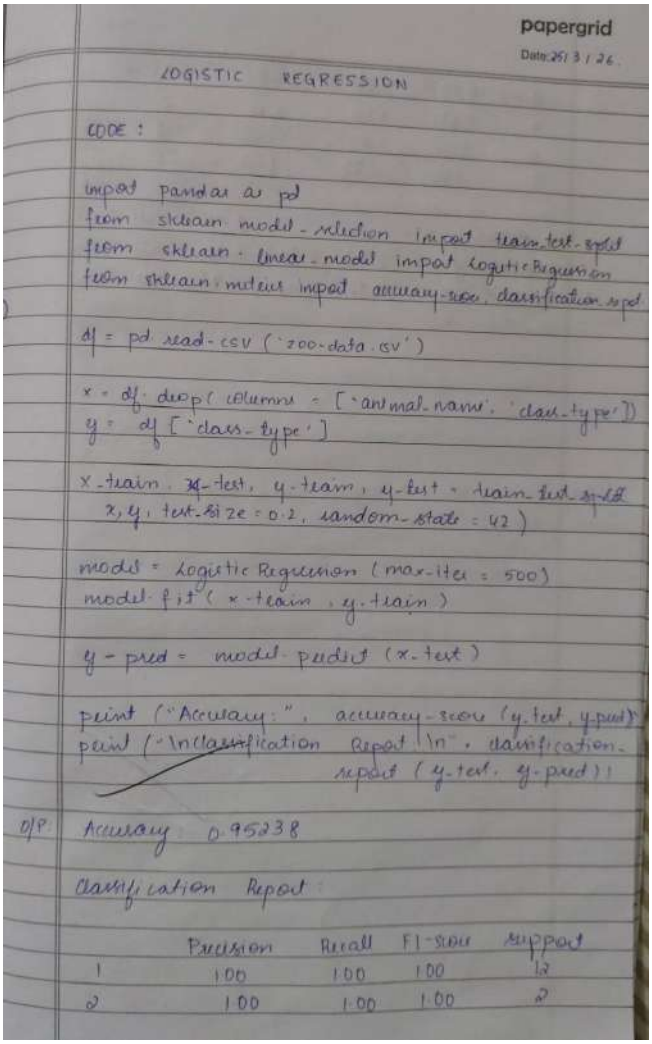
```python
# ================================
# CONFUSION MATRIX
# ================================
def confusion_matrix(y_true, y_pred):
    classes = np.unique(y_true)
    matrix = np.zeros((len(classes), len(classes)), dtype=int)

    for t, p in zip(y_true, y_pred):
        matrix[t][p] += 1

    return matrix

# ================================
# PREPARE DATA
# ================================
X = df.iloc[:, :-1].values   # features
y, label_map = encode_labels(df.iloc[:, -1].values)  # labels

X = normalize(X)

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y)

# ================================
# TRAIN & TEST
# ================================
k = 5
y_pred = knn_predict(X_train, y_train, X_test, k)

# ================================
# RESULTS
# ================================
print("\nLabel Mapping:", label_map)
print("K:", k)
print("Accuracy:", accuracy(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Dataset Preview:
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species
0   1            5.1           3.5            1.4           0.2  Iris-setosa
1   2            4.9           3.0            1.4           0.2  Iris-setosa
2   3            4.7           3.2            1.3           0.2  Iris-setosa
3   4            4.6           3.1            1.5           0.2  Iris-setosa
4   5            5.0           3.6            1.4           0.2  Iris-setosa

Label Mapping: {'Iris-setosa': 0, 'Iris-virginica': 1, 'Iris-versicolor': 2}
K: 5
Accuracy: 1.0

Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

Program 4

Implement Logistic Regression algorithm using appropriate dataset

Screenshot :



papergrid

Date: 2023/26

LOGISTIC REGRESSION

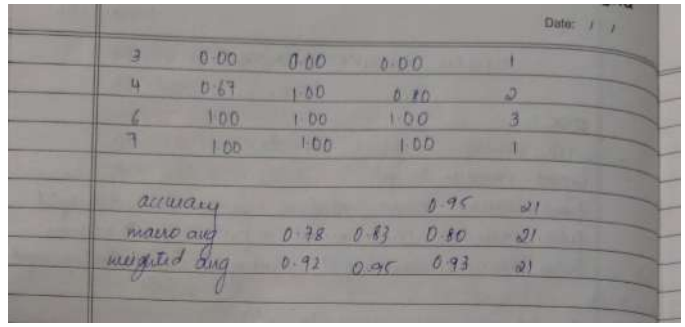
CODE :

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

df = pd.read_csv('zoo-data.csv')

x = df.drop(columns = ['animal_name', 'class_type'])
y = df['class_type']

x_train, x_test, y_train, y_test = train_test_split
    x, y, test_size = 0.2, random_state = 42)

model = LogisticRegression(max_iter = 500)
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report\n", classification_report(y_test, y_pred))
```

O/P: Accuracy: 0.95238

Classification Report:

	Precision	Recall	F1-score	support
1	1.00	1.00	1.00	12
2	1.00	1.00	1.00	2



Date: / /

3	0.00	0.00	0.00	1
4	0.67	1.00	0.80	2
6	1.00	1.00	1.00	3
7	1.00	1.00	1.00	1
accuracy			0.95	21
macro avg	0.78	0.83	0.80	21
weighted avg	0.92	0.95	0.93	21

Code:

In [1]:
```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler
```

In [2]:
```
data = load_breast_cancer()
```

In [3]:
```
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target

print(df.head())
```

```
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0        17.99         10.38          122.80     1001.0          0.11840
1        20.57         17.77          132.90     1326.0          0.08474
2        19.69         21.25          130.00     1203.0          0.10960
3        11.42         20.38           77.58      386.1          0.14250
4        20.29         14.34          135.10     1297.0          0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry  \
0           0.27760          0.3001              0.14710         0.2419
1           0.07864          0.0869              0.07017         0.1812
2           0.15990          0.1974              0.12790         0.2069
3           0.28390          0.2414              0.10520         0.2597
4           0.13280          0.1980              0.10430         0.1809

   mean fractal dimension  ...  worst texture  worst perimeter  worst area  \
0                 0.07871  ...          17.33           184.60      2019.0
1                 0.05667  ...          23.41           158.80      1956.0
2                 0.05999  ...          25.53           152.50      1709.0
3                 0.09744  ...          26.50            98.87       567.7
4                 0.05883  ...          16.67           152.20      1575.0

   worst smoothness  worst compactness  worst concavity  worst concave points  \
0            0.1622             0.6656           0.7119                0.2654
1            0.1238             0.1866           0.2416                0.1860
2            0.1444             0.4245           0.4504                0.2430
3            0.2098             0.8663           0.6869                0.2575
4            0.1374             0.2050           0.4000                0.1625
```

In [4]:
```
df = df[['mean radius', 'mean texture', 'mean perimeter', 'mean area', 'mean smoothness']]
```

File display

```
df['target'] = data.target
```

In [6]:
```
print(df.head())
```

```
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0        17.99         10.38          122.80     1001.0          0.11840
1        20.57         17.77          132.90     1326.0          0.08474
2        19.69         21.25          130.00     1203.0          0.10960
3        11.42         20.38           77.58      386.1          0.14250
4        20.29         14.34          135.10     1297.0          0.10030

   target
0       0
1       0
2       0
3       0
4       0
```

In [7]:
```
X = df.drop('target', axis=1)
y = df['target']
```

In [8]:
```
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

In [9]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

In [10]:
```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

Out[10]: LogisticRegression(max_iter File display

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [11]:
```
y_pred = model.predict(X_test)
```

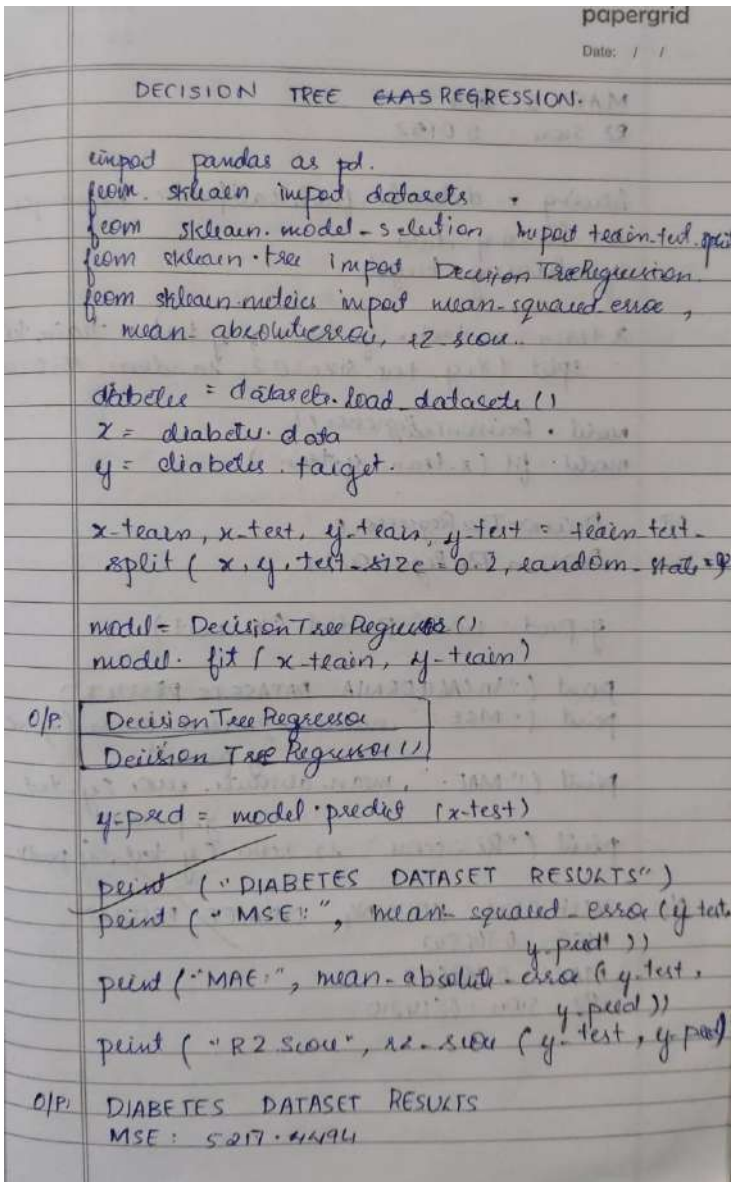
In [12]:
```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.9385964912280702

Confusion Matrix:
 [[39  4]
 [ 3 68]]

Classification Report:
               precision    recall  f1-score   support

           0       0.93      0.91      0.92        43
           1       0.94      0.96      0.95        71

    accuracy                           0.94       114
   macro avg       0.94      0.93      0.93       114
weighted avg       0.94      0.94      0.94       114
```

Program 5

Decision Tree Classification & Regression :

Screenshot :



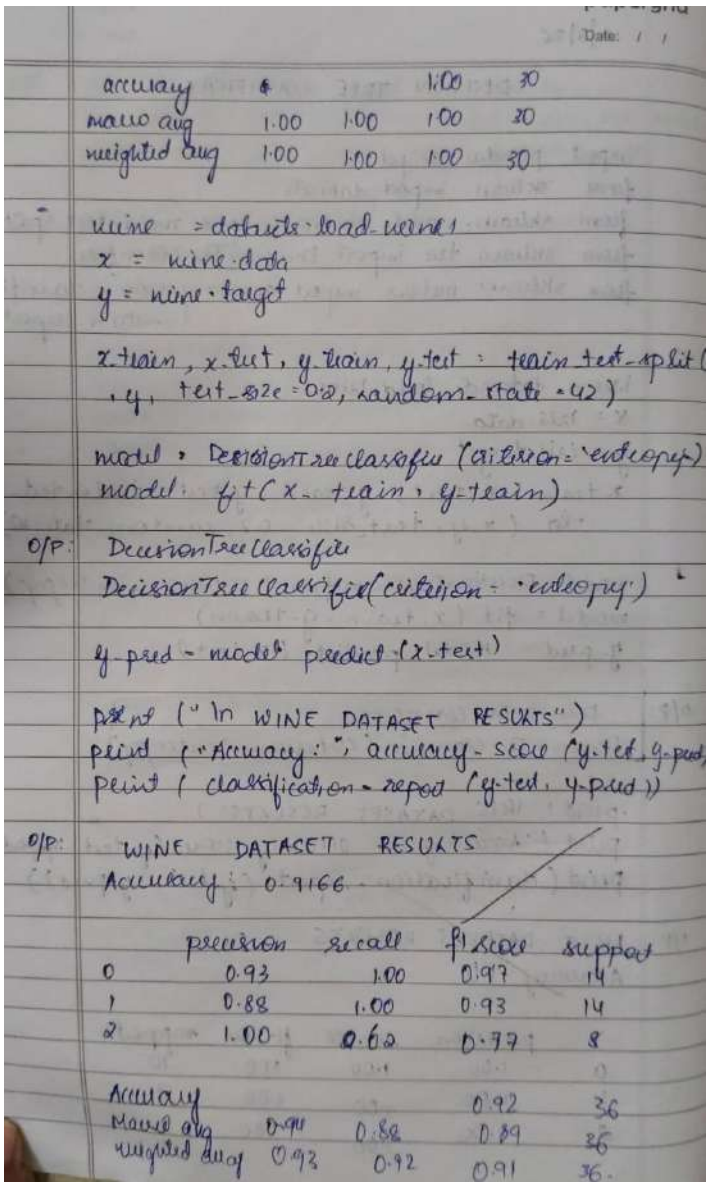
DECISION TREE ~~CLAS~~ REGRESSION.

```
import pandas as pd.
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error,
    mean_absolute_error, r2_score.

diabetes = datasets.load_datasets()
x = diabetes.data
y = diabetes.target.

x_train, x_test, y_train, y_test = train_test_
    split( x, y, test_size = 0.2, random_state = 4
model = DecisionTreeRegressor()
model.fit( x_train, y_train)
```

O/P: DecisionTreeRegressor
DecisionTreeRegressor()

```
y_pred = model.predict (x_test)

print ("DIABETES DATASET RESULTS")
print ("MSE:", mean_squared_error (y_test,
                                   y_pred))
print ("MAE:", mean_absolute_error (y_test,
                                   y_pred))
print ("R2 Score", r2_score (y_test, y_pred)
```

O/P: DIABETES DATASET RESULTS
MSE : 5217.4494



accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

```
wine = datasets.load_wine()
x = wine.data
y = wine.target

x_train, x_test, y_train, y_test = train_test_split(
    , y, test_size = 0.2, random_state = 42)

model = DecisionTreeClassifier (criterion = 'entropy')
model.fit( x_train, y_train)
```

O/P: DecisionTreeClassifier
DecisionTreeClassifier(criterion = 'entropy')

```
y_pred = model.predict (x_test)

print ("In WINE DATASET RESULTS")
print ("Accuracy:", accuracy_score (y_test, y_pred)
print ( classification_report (y_test, y_pred))
```

O/P: WINE DATASET RESULTS
Accuracy: 0.9166

	precision	recall	f1 score	support
0	0.93	1.00	0.97	14
1	0.88	1.00	0.93	14
2	1.00	0.62	0.77	8
Accuracy			0.92	36
Macro avg	0.94	0.88	0.89	36
weighted avg	0.93	0.92	0.91	36

papergrid

Date: / /

DECISION TREE ~~CLAS~~ REGRESSION.

```
import pandas as pd.
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error,
    mean_absolute_error, r2_score.

diabetes = datasets.load_datasets()
x = diabetes.data
y = diabetes.target.

x_train, x_test, y_train, y_test = train_test_
    split (x, y, test_size = 0.2, random_state = 9

model = DecisionTreeRegressor()
model.fit (x_train, y_train)
```

O/P: DecisionTreeRegressor
DecisionTreeRegressor()

```
y_pred = model.predict (x_test)

print ("DIABETES DATASET RESULTS")
print ("MSE:", mean_squared_error (y_test,
                                   y_pred))
print ("MAE:", mean_absolute_error (y_test,
                                    y_pred))
print ("R2 Score", r2_score (y_test, y_pred)
```

O/P: DIABETES DATASET RESULTS
MSE: 5217.44944

papergrid

Date: / /

MAE: 55.814
R2 Score: 0.0152

```
housing = datasets.fetch_california_housing()
x = housing.data
y = housing.target

x_train, x_test, y_train, y_test = train_test_
    split (x, y, test_size = 0.2, random_state =

model = DecisionTreeRegressor()
model.fit (x_train, y_train).
```

O/P: DecisionTreeRegressor
DecisionTreeRegressor()

```
y_pred = model.predict (x_test)

print ("In CALIFORNIA DATASETS RESULTS")
print ("MSE:", mean_squared_error (y_test,
                                   y_pred))
print ("MAE:", mean_absolute_error (y_test,
                                    y_pred))
print ("R2 Score", r2_score (y_test, y_pred)
```

O/P: CALIFORNIA HOUSING DATASET RESULTS
MSE: 0.505542
MAE: 0.457607
R2 Score: 0.614210.

Code :

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import mean_squared_error, r2_score

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import DecisionTreeRegressor
```

```
from sklearn.datasets import load_iris

data = load_iris()

X = data.data
y = data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
model = DecisionTreeClassifier(criterion='entropy')

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

print(classification_report(y_test, y_pred))
```

```
Accuracy: 1.0
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

```
from sklearn.datasets import load_wine

data = load_wine()

X = data.data
y = data.target
```

```
from sklearn.datasets import load_diabetes

data = load_diabetes()

X = data.data
y = data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
regressor = DecisionTreeRegressor()

regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)
```

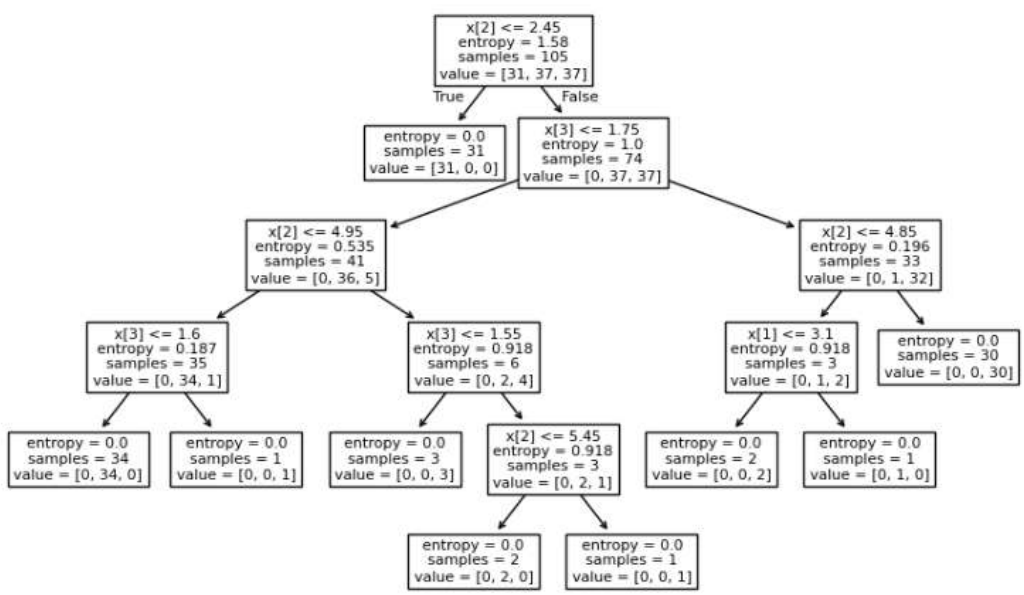
```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("R2 Score:", r2)
```

```
Mean Squared Error: 5358.458646616541
R2 Score: 0.0073817790735694055
```

```
from sklearn.datasets import fetch_california_housing

data = fetch_california_housing()

X = data.data
y = data.target
```

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
plot_tree(model)
plt.show()
```

...



Program 6

Support Vector Machine :

Screenshot :

Date: / /

SVM.

```
import pandas as pd
from sklearn.datasets import load_breast_cancer.
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

data = load_breast_cancer()
x = data.data
y = data.target

x_train, x_test, y_test, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
model = SVC(kernel='linear')
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\n Classification Report:\n", classification_report(y_test, y_pred))
```

Accuracy: 0.956140

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.91	0.96	43
1	0.95	0.99	0.97	71
accuracy			0.96	114
macro avg	0.96	0.95	0.95	114
weighted avg	0.96	0.96	0.96	114

Code:

```python
# Import libraries
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler

# Load dataset
data = datasets.load_iris()
X = data.data
y = data.target

# Feature scaling (important for SVM)
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split dataset (fixed for same results every time)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create SVM model
model = SVC(kernel='linear')

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
```

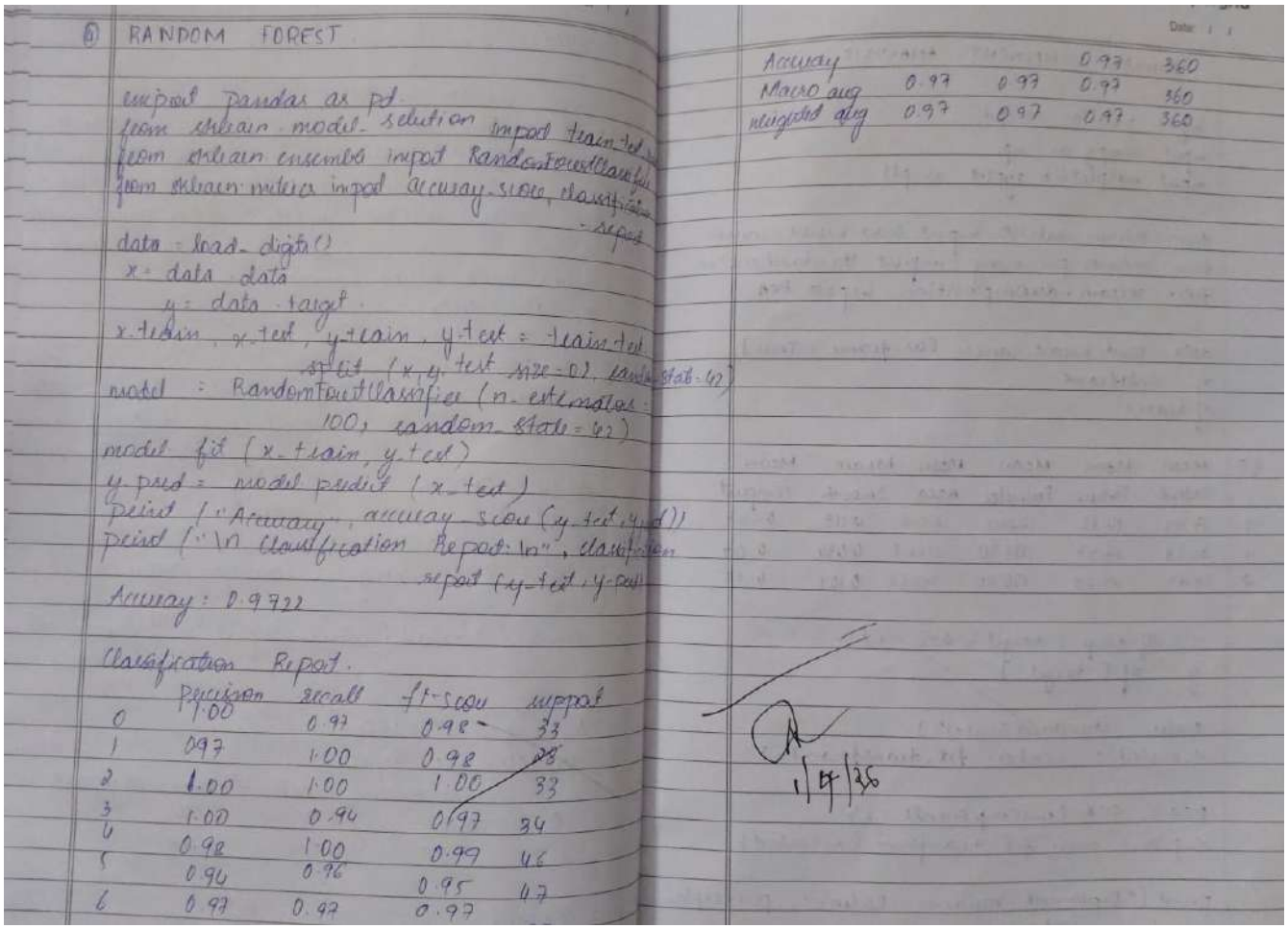
```python
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)

# Classification Report
cr = classification_report(y_test, y_pred)
print("\nClassification Report:\n", cr)
```

```
Accuracy: 0.9666666666666667

Confusion Matrix:
 [[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]

Classification Report:
               precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      0.89      0.94         9
           2       0.92      1.00      0.96        11

    accuracy                           0.97        30
   macro avg       0.97      0.96      0.97        30
weighted avg       0.97      0.97      0.97        30
```

Program 7

Random Forest :

Screenshot :



6 RANDOM FOREST

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
data = load_digits()
x = data.data
y = data.target
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
print("Accuracy", accuracy_score(y_test, y_pred))
print("\n Classification Report:\n", classification_report(y_test, y_pred))
```

Accuracy : 0.9722

Classification Report.

	Precision	recall	f1-score	support
0	1.00	0.97	0.98	33
1	0.97	1.00	0.98	33
2	1.00	1.00	1.00	33
3	1.00	0.94	0.97	34
4	0.98	1.00	0.99	46
5	0.96	0.96	0.95	47
6	0.97	0.97	0.97	
Accuracy			0.97	360
Macro avg	0.97	0.97	0.97	360
weighted avg	0.97	0.97	0.97	360

1/4/26

Code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(n_estimators=100, random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
Accuracy: 1.0
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

Program 8

ADA Boost Classifier :

Screenshot :

LAB-6

papergrid

Date: / /

BOOSTING ENSEMBLE LEARNING.

① ADABOOST CLASSIFIER.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()
x = data.data
y = data.target

x_train, x_test, y_train, y_test = train_test_split(
                x, y, test_size=0.2, random
                _state=42)
model = AdaBoostClassifier(n_estimators=50, learning
                _rate=1.0, random_state=42).
model.fit(x_training, y_train)
y_pred = model.predict(x_test)
print("Accuracy:", accuracy_score(y_test,
                y_pred))
```

o/p Accuracy: 0.9649122

classification Report:

	precision	recall	f1-score	support
0	0.98	0.93	0.95	43
1	0.96	0.99	0.97	71
Accuracy				
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

Code :

```python
# Import libraries
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load Wine dataset
data = load_wine()
X = data.data
y = data.target

# Split dataset into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create AdaBoost model
model = AdaBoostClassifier(
    n_estimators=50,      # number of weak learners
    learning_rate=1.0,    # contribution of each classifier
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```python
# Evaluate model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.9444444444444444

Classification Report:
               precision    recall  f1-score   support

           0       0.93      1.00      0.97        14
           1       0.93      0.93      0.93        14
           2       1.00      0.88      0.93         8

    accuracy                           0.94        36
   macro avg       0.95      0.93      0.94        36
weighted avg       0.95      0.94      0.94        36
```

Program 9

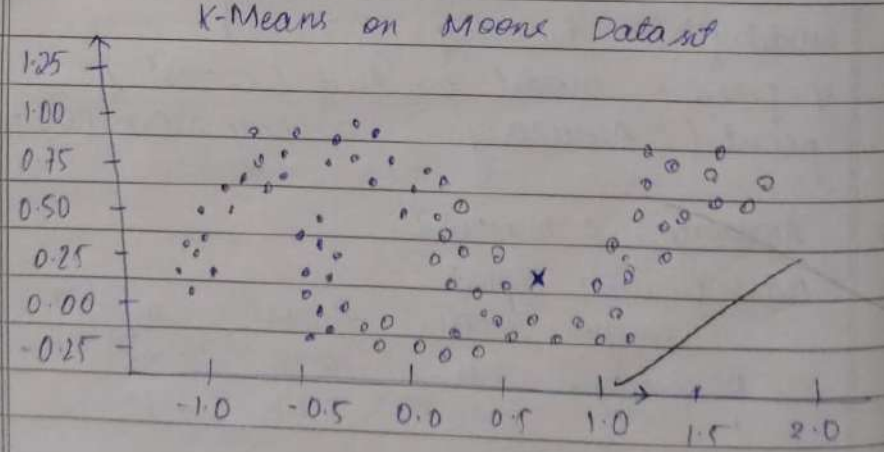
K - Means Algorithm :

Screenshot :

Date: / /

② K-MEANS ALGORITHM.

```
import matplotlib.pyplot as plt.
from sklearn.datasets import make_moons
from sklearn.cluster import KMeans.

x, y = make_moons(n_samples = 300, noise = 0.1,
                  random_state = 42)
kmeans = KMeans(n_clusters = 2, random_state=
kmeans.fit(x)
labels = kmeans.labels_
plt.scatter(x[:,0], x[:,1], c = labels)
plt.scatter(kmeans.cluster_centers_[:,0], kmeans
            cluster_centers[:,1], marker='X')
plt.title("K-Means on Moons Dataset")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```



Code :

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

```
data = pd.read_csv("/content/data.csv")
print(data.head())
```

```
X = data.iloc[:, :].values   # or choose specific columns
```

```
k = 3   # you can change this
```

```
kmeans = KMeans(n_clusters=k, random_state=0)
kmeans.fit(X)
```

```
labels = kmeans.labels_
print(labels)
```

```
centroids = kmeans.cluster_centers_
print(centroids)
```

```
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.scatter(centroids[:, 0], centroids[:, 1], s=200, c='red')
plt.title("K-Means Clustering")
plt.show()
```

Program 11

PCA :

Screenshot :

PRINCIPAL COMPONENT ANALYSIS.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt.

from sklearn.datasets import load_breast_cancer.
from sklearn.preprocessing import standard scaler
from sklearn.decomposition import PCA

data = load_breast_cancer(as_frame = True)
df = dataframe
df.head()
```

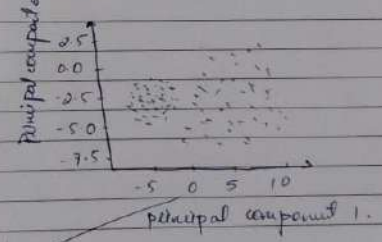
O/P

	Mean radius	Mean Texture	Mean Perimeter	Mean Area	Mean Smooth	Mean compact
0	17.99	10.38	122.80	1001.0	0.118	0.27
1	20.57	17.77	132.90	1326.0	0.084	0.07
2	19.69	21.25	130.00	1203.0	0.109	0.15

```
x = df.drop('target', axis=1)
y = df['target']

scaler = standardScaler()
x_scaled = scaler.fit_transform(x)

pca = PCA(n_components=2)
x_pca = pca.fit_transform(x_scaled)

print("Explained variance Ratio:", pca.explained_variance_ratio_)
```

```
pca_df = pd.DataFrame(data = x_pca, columns = ['pc1', 'pc2'])
pca_df['target'] = y.

plt.figure()
for label in pca_df['target'].unique():
    subset = pca_df[pca_df['target'] == label]
    plt.scatter(subset['pc1'], subset['pc2'])
    label = label()
plt.xlabel("Principal component 1")
plt.ylabel("Principal component 2")
plt.title("PCA visualization")
plt.legend()
plt.show()
```

O/P:



15/4/26

Code:

PCA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

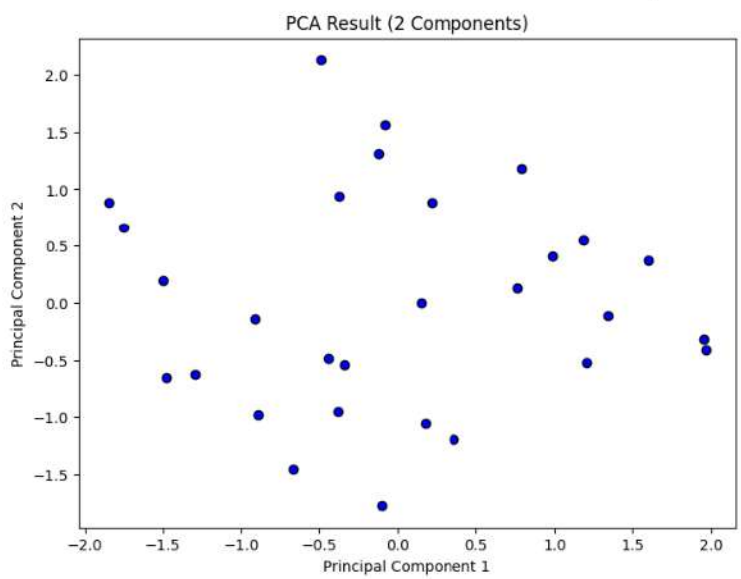
```
# Load dataset
data = pd.read_csv("data.csv")
print(data.head())
```

```
    x   y
0  30  49
1  31  27
2  31  46
3   1  65
4  81  76
```

```
# Standardize the data
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)
```

```
# Apply PCA
pca = PCA(n_components=2)  # Reduce to 2 components
pca_result = pca.fit_transform(scaled_data)
```

```
# Explained variance ratio
print(f"Explained Variance Ratio: {pca.explained_variance_ratio_}")
print(f"Total Variance Explained by 2 Components: {np.sum(pca.explained_variance_ratio_)}")
```

```
Explained Variance Ratio: [0.56506606 0.43493394]
Total Variance Explained by 2 Components: 1.0
```

```
# Scatter plot of the 2D PCA result
plt.figure(figsize=(8, 6))
plt.scatter(pca_result[:, 0], pca_result[:, 1], c='blue', edgecolors='black')
plt.title('PCA Result (2 Components)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()
```